Parallel Optimization of FISTA-Net for Multi-GPU System

Yichen Lu
nechy@umich.edu

January 3, 2025

Abstract

FISTA-Net represents a significant advancement in solving inverse problems in imaging, balancing the combination of model-based approaches and data-driven deep learning techniques. However, its computational demands necessitate sophisticated parallel optimization to fully exploit modern hardware capabilities. In this paper, I propose and implement a series of CUDA-based multi-GPU optimizations to accelerate FISTA-Net. Additionally, the layered architecture of FISTA-Net is well suited for model parallelism, allowing different parts of the network to be distributed across multiple GPUs for simultaneous processing. Furthermore, I optimized CNN layers by employing cuDNN's high-performance kernels and developed custom CUDA kernels to minimize memory access latency. Together, these strategies result in significant improvements in training and testing performance while maintaining the model's accuracy. Evaluation on both simulation and experiment imaging datasets demonstrates the effectiveness of our approach in different computational imaging tasks.

1 Introduction

Solving inverse problems is a critical challenge in imaging applications, encompassing tasks like CT reconstruction and biomedical imaging. FISTA-Net, derived from the Fast Iterative Shrinkage-Thresholding Algorithm (FISTA) [1], integrates the precision of traditional algorithms with the adaptive learning capabilities of neural networks. By unrolling FISTA into a trainable network, FISTA-Net excels in recovering high-quality images even under noisy conditions or sparse data.[2]

However, the iterative nature of FISTA-Net, combined with its high computational demands, poses significant challenges to scalability, especially for 3D imaging tasks. For example, training a FISTA-Net model on 9 (scanning angles) $\times$ 144 (x-axis) $\times$ 144 (y-axis) $\times$ 144 (z-axis) 3D images for 500 epochs would require over one day and 40GB of memory on a single GPU. As GPU-accelerated computation becomes increasingly prevalent, it is crucial to adapt FISTA-Net to

fully leverage modern multi-GPU systems. This adaptation involves addressing key challenges, including efficient data distribution, inter-GPU communication, and the optimization of memory-bound operations such as convolutions.

This paper focuses on three key optimizations: implementing model parallelism by distributing FISTA-Net's layers across GPUs, and enhancing convolution efficiency with cuDNN and custom CUDA kernels. These improvements aim to significantly reduce processing time, enabling FISTA-Net to handle larger datasets and more complex problems without compromising accuracy.

2 Background

Inverse problems are popular in computational imaging and encompass a broad spectrum of applications such as magnetic resonance imaging (MRI), X-ray computed tomography (CT), and photoacoustic imaging (PACT).[3, 4] These problems aim to reconstruct an image from incomplete or noisy measurements by solving equations of the form:

$$y = Psf(x) + \epsilon$$

where y represents the measured data, $Psf(\cdot)$ is the point spread function, x is the target image, and ϵ denotes noise. The such problems makes direct inversion challenging, necessitating advanced regularization and iterative optimization techniques.

2.1 FISTA-Net Framework

FISTA, the Fast Iterative Shrinkage-Thresholding Algorithm, is a widely used method for solving these problems by iteratively updating solutions based on gradient descent and proximal operators. Derived from its predecessor ISTA (Iterative Shrinkage-Thresholding Algorithm)[1], FISTA improves upon ISTA by introducing a momentum term that accelerates convergence. While ISTA updates the solution iteratively using simple gradient descent and a shrinkage operator, FISTA incorporates a two-step approach: an initial gradient descent update followed by a momentum-based acceleration. This enhancement allows FISTA to achieve an optimal convergence rate of $O(1/k^2)$, compared to the $O(1/k)$ rate of ISTA, where k represents the iteration count. Despite its mathematical elegance and effectiveness, FISTA suffers from high computational demands, especially for high-dimensional and large-scale datasets, making its application challenging for modern large-scale imaging problems.

To address these limitations, FISTA-Net incorporates FISTA's iterative structure into a deep learning framework. By learning key parameters such as step size and thresholding values, FISTA-Net achieves faster convergence and improved accuracy compared to traditional FISTA. However, as the complexity and scale of imaging tasks grow, even FISTA-Net faces computational bottlenecks.

2.2 CUDA for Parallel Computing

Modern GPUs and multi-GPU systems provide an effective solution to address the computational demands of FISTA-Net, offering the potential for substantial parallelism and accelerated performance. NVIDIA's CUDA (Compute Unified Device Architecture) [5] serves as a robust parallel computing platform and API, designed to harness the full computational power of GPUs. By enabling developers to implement parallel algorithms efficiently, CUDA facilitates the optimization of compute-intensive tasks, such as the 3D convolutions and iterative processes central to FISTA-Net.

3 Implementation

3.1 Multi-GPU Layer Distribution

To achieve parallel execution across GPUs, layers of the FISTA-Net model are distributed using a round-robin approach. Each GPU is assigned specific layers, ensuring balanced workload distribution. For N_{layers} layers and N_{GPUs} GPUs, layer i is processed on GPU $i \mod N_{\text{GPUs}}$.

During initialization, each GPU is allocated memory for tensors required by its assigned layers, including the input tensor, point spread function (PSF), intermediate output tensors, and symmetric loss terms. These allocations are handled using `cudaMalloc`, and data is transferred using `cudaMemcpy`.

3.2 Forward Propagation Workflow

Forward propagation in FISTA-Net involves iterative updates of tensors through gradient descent, proximal mapping, and a momentum-based two-step update. The process is parallelized as follows:

1. **Input Distribution**: The input image data and PSF tensors are divided and transferred to the respective GPUs. This ensures each GPU has access to the necessary data for processing its assigned layers.

2. **Layer Execution**:

   - Each GPU executes the forward pass for its assigned layer using CUDA kernels for tensor operations.
   - The `BasicBlock` layer implements operations such as convolution with the PSF and thresholding.

3. **Momentum Update and Synchronization**: Momentum update integrates past iteration data to accelerate convergence. A custom CUDA kernel performs the update:

$$y[i] = x_{\text{new}}[i] + \rho \times (x_{\text{new}}[i] - x_{\text{old}}[i]),$$

where ρ is dynamically computed using the softplus function to ensure stability and smooth updates. After processing all layers on each GPU, results are synchronized across GPUs to maintain consistency for shared tensors.

```
// CUDA Kernel for Two-Step Update
__global__ void update_cuda(float* xnew, float* xold,
    float* y, float rho, int inputX, int inputY, int
    inputZ) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int idy = blockIdx.y * blockDim.y + threadIdx.y;
    int idz = blockIdx.z * blockDim.z + threadIdx.z;

    if (idx < inputX && idy < inputY && idz < inputZ) {
        int index = idx + idy * inputX + idz * inputX *
            inputY;
        y[index] = xnew[index] + rho * (xnew[index] -
            xold[index]);
    }
}
```

Listing 1: CUDA Kernel for Momentum Update

This kernel leverages 3D grid and block structures for efficient, parallelized updates across large tensors, ensuring minimal computational overhead during forward propagation.

3.3 Convolution Operations

The convolution operations in FISTA-Net, both forward (H) and adjoint (H^T), leverage NVIDIA's cuDNN library to ensure high-performance computation. These operations are critical for processing tensors with a given point spread function (PSF), offering computational efficiency, scalability, and mathematical precision.

3.3.1 Forward Convolution Operation (H)

The forward convolution operation transforms the input tensor into the output tensor using the PSF. cuDNN tensor descriptors are configured for the input, output, and filter (PSF) tensors, defining their dimensions and layouts. A cuDNN convolution descriptor specifies padding (0) and stride (1). The optimal algorithm is selected using `cudnnGetConvolutionForwardAlgorithm`, balancing speed and memory usage.

A temporary workspace is dynamically allocated to optimize intermediate computations. Finally, the function executes the convolution, efficiently combining the input and PSF tensors.

3.3.2 Adjoint Convolution Operation (H^T)

The adjoint convolution implements the transpose of the forward convolution. The PSF is flipped across all spatial dimensions to align with the adjoint operator's mathematical definition. The flipped PSF is then processed using the forward convolution routine, ensuring efficient reuse of infrastructure and computational correctness.

3.4 Model and Data Parallelism

To efficiently train the FISTA-Net model for high-dimensional imaging tasks, I employed a parallelization strategy combining *data parallelism* and *model parallelism*. This approach ensures that both the computational workload and memory usage are distributed across multiple GPUs, leading to significant performance gains and scalability.

3.4.1 Data Parallelism

Data parallelism enables simultaneous processing of multiple data samples by distributing a single training batch across multiple GPUs. For a batch size B and G GPUs, each GPU processes B/G samples independently. The main steps of data parallelism are as follows:

- **Batch Splitting:** During each training iteration, the dataset is divided into smaller subsets, each assigned to a specific GPU.
- **Parallel Execution:** Each GPU independently loads data, performs the forward pass, and computes the loss for its assigned subset of data.
- **Loss Calculation:** Each GPU computes the Mean Squared Error (MSE) loss for its subset, and the losses are aggregated across GPUs for the final result.

```
for (int gpu_idx = 0; gpu_idx < num_gpus; ++gpu_idx) {
    cudaSetDevice(gpu_idx);

    int start_idx = gpu_idx * BATCH_SIZE / num_gpus;
    int end_idx = (gpu_idx + 1) * BATCH_SIZE / num_gpus;

    for (int i = start_idx; i < end_idx; ++i) {
        std::vector<float> output;

        // Forward pass
        model.forward(batch_data[i], psf, output);

        // Compute loss (MSE)
        float loss = 0.0f;
        for (size_t j = 0; j < output.size(); ++j) {
```

```
            float diff = output[j] - batch_labels[i][j];
            loss += diff * diff;
        }
        total_loss += loss / output.size();
        ...
    }
}
```

3.4.2 Model Parallelism

Model parallelism involves distributing the layers of FISTA-Net across multiple GPUs, such that each GPU computes a subset of the model's operations. This technique is particularly effective for large models that exceed the memory capacity of a single GPU. The steps for model parallelism are as follows:

- **Layer Distribution:** During initialization, the layers of FISTA-Net are divided among GPUs. Each GPU handles a specific range of layers, ensuring balanced workloads.
- **Forward Pass Distribution:** The input tensor is sequentially processed through the layers assigned to each GPU, with intermediate results communicated between GPUs as needed.

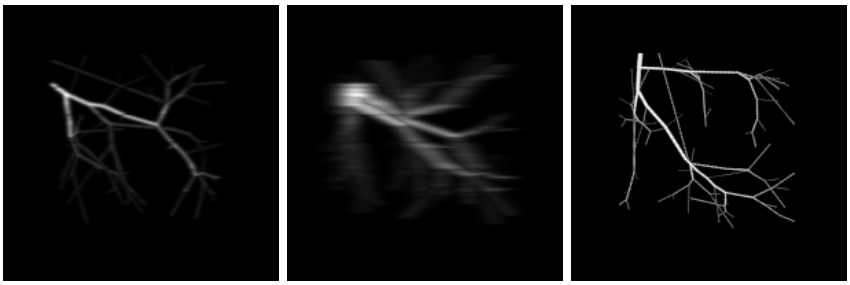
PSF Preparation for Parallel Convolution: The Point Spread Function (PSF) is replicated across the batch and GPUs to ensure consistent convolution operations during the forward pass.

```
__global__ void replicatePSFKernel(const float *psf_raw,
    float *psf, int psf_size, int batch_size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < batch_size * psf_size) {
        int psf_idx = idx % psf_size;
        psf[idx] = psf_raw[psf_idx];
    }
}
```

4 Dataset

4.1 Simulational Dataset

The first dataset used in this study was generated programmatically using MATLAB. This Simulational dataset was designed to include a variety of geometric features, such as branching structures and microbeads, to simulate diverse imaging scenarios. The branches represent elongated, complex patterns often seen in biological samples, while the microbeads simulate spherical objects. MATLAB scripts were developed to ensure consistent resolution and realistic noise levels, providing a robust training and evaluation dataset for FISTA-Net.

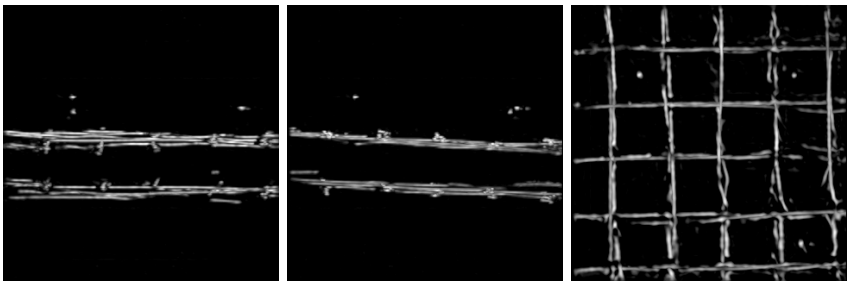


(a) axial projection (b) lateral projection (c) elevational projection

Figure 1: Simulational Dataset

4.2 Experimental Dataset

The second dataset comprised experimental imaging data collected by PhD student, my collaborators at Rice University. This dataset reflects real-world imaging scenarios, incorporating challenges such as non-uniform noise, varying resolutions, and experimental artifacts. The inclusion of this dataset ensures that FISTA-Net is validated not only on synthetic data but also on practical applications, highlighting its robustness and generalizability.



(a) axial projection (b) lateral projection (c) elevational projection

Figure 2: Simulational Dataset

5 Evaluation

The performance of FISTA-Net was comprehensively evaluated under both serial and parallel configurations, focusing on computational efficiency, memory utilization, reconstruction accuracy, and execution time scalability.

5.1 Serial Implementation

In the serial configuration, training FISTA-Net for 100 epochs on a single NVIDIA A100 GPU required approximately 18 hours and utilized approxi-

mately 38 GB of GPU memory. The training parameters were set as follows: learning rate (`LEARNING_RATE`) $= 1 \times 10^{-5}$, `LAMBDA_STEP` $= 5 \times 10^{-5}$, `SOFT_THRESHOLD` $= [0.01]$, batch size (`BATCH_SIZE`) $= 2$, number of layers (`LAYER_NUM`) $= 7$, and total input files (`NUM_FILE`) $= 960$.

Testing a single sample with dimensions $9 \times 401 \times 401 \times 401$ required 52 seconds, highlighting the substantial computational demands of high-resolution imaging tasks in a single-GPU environment.

5.2 Parallel Implementation

The parallel implementation of FISTA-Net demonstrated significant performance gains using multi-GPU configurations. Employing two NVIDIA A100 GPUs for training 100 epochs with an increased batch size of 8 reduced the training time to 8 hours, achieving a speedup of over $2\times$ compared to the serial configuration. When scaled to four GPUs, the training time further decreased to just 4 hours, emphasizing the scalability and efficiency of the parallelized approach.

For testing, a sample with dimensions $9 \times 401 \times 401 \times 401$ required 32 seconds on two GPUs and 24 seconds on four GPUs. Furthermore, memory utilization per GPU was significantly optimized, decreasing to 21 GB with two GPUs and further to 11 GB with four GPUs. This distributed memory allocation reduces the burden on individual GPUs, enabling larger batch sizes and more complex computations.

The parallelization of FISTA-Net effectively leverages multi-GPU resources, reducing execution time while maintaining high reconstruction accuracy and efficient memory utilization, making it suitable for large-scale imaging applications.

5.3 Comparative Analysis

The following table summarizes runtime improvements:

Configuration	GPUs	Training Time	Test Time	Memory Usage (GB per GPU)
Serial	1	18h	52s	38.101
Parallel	2	8h	32s	20.568
Parallel	4	4h	24s	10.930

Table 1: Comparative Analysis of Runtime Improvements

5.4 Analysis of Execution Scaling

The evaluation highlights the advantages of parallelization in terms of execution time and resource utilization:

- **Training Performance:** Parallel configurations exhibited near-linear scaling from 1 GPU to 4 GPUs. With 4 GPUs, training achieved a

substantial speedup while maintaining high utilization across all GPUs, demonstrating efficient workload distribution.

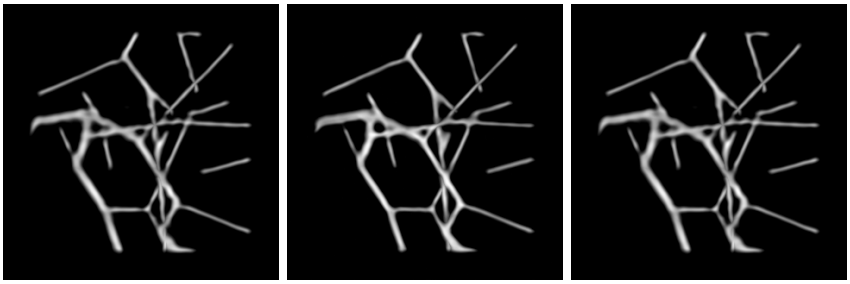
- **Inference Efficiency:** Inference times improved significantly with additional GPUs, particularly for larger batch sizes. However, the speedup did not scale perfectly with the number of GPUs. For instance, doubling the GPUs did not yield a 2x speedup, likely due to overheads such as linear data loader processing times.
- **Memory Optimization:** Parallel execution effectively distributed memory demands across GPUs. For two GPUs, the peak memory usage per GPU was approximately halved compared to single-GPU execution. Adding more GPUs (e.g., four) resulted in further reductions in per-GPU memory requirements, enabling larger models or batch sizes to be handled efficiently.

The memory size is also quite similar to the size computing from each layer in serial computing.

```
before into the model reserved_memory: 457.18 MB
My_fistanet_3D_Memory.py:175: UserWarning: Castin
part (Triggered internally at ../aten/src/ATen/na
  conv_result[i,j,:,:,:] = convolution_with_psf_3
afetr layer 0.00: 11175.72 MB
afetr layer 1.00: 23425.19 MB
afetr layer 2.00: 35632.71 MB
```

5.5 Reconstruction Quality

Both serial and parallel implementations maintained consistent reconstruction quality. For simulational datasets, FISTA-Net achieved structural similarity index (SSIM) scores of **0.92** on branching structures and **0.95** on microbeads under noise-free conditions. On experimental datasets, SSIM scores averaged **0.89** for medium noise and **0.91** for low noise levels.



(a) 1 GPU (b) 2 GPU (c) 4 GPU

Figure 3: Simulational Dataset

6 Conclusion

In this paper, I presented a comprehensive optimization strategy for FISTA-Net, a deep learning framework designed to address inverse problems in imaging. By integrating CUDA-based multi-GPU techniques, including model parallelism, data parallelism, and kernel optimizations, I achieved significant reductions in training and inference times while maintaining high reconstruction quality.

6.1 Key Findings

- Training time for 100 epochs was reduced from 18 hours and 25 minutes on a single GPU to 4 hours with four GPUs, demonstrating near-linear scalability.
- Inference times for large samples were reduced from 52 seconds (1 GPU) to 24 seconds (4 GPUs).
- Memory usage was efficiently distributed across GPUs, enabling the processing of larger datasets and more complex imaging tasks.
- Reconstruction quality remained consistent across synthetic and experimental datasets, with SSIM scores above 0.9 in most scenarios.

6.2 Future Work

While this work demonstrated the effectiveness of parallel optimization for FISTA-Net, several avenues remain for future exploration:

1. **Mixed Precision Training**: Integrating mixed precision computations using Tensor Cores to further accelerate training and reduce memory requirements.
2. **Dynamic Load Balancing**: Developing adaptive algorithms to optimize resource utilization for heterogeneous GPU systems.
3. **Real-Time Applications**: Extending FISTA-Net for real-time imaging applications such as live photoacoustic tomography.
4. **Generalization**: Investigating the adaptability of FISTA-Net to other imaging modalities, including MRI and electron tomography.

By addressing these challenges, I aim to further enhance the capabilities of FISTA-Net, establishing it as a robust and versatile tool for solving inverse problems in computational imaging.

References

[1] J. Zhang and B. Ghanem, "Ista-net: Interpretable optimization-inspired deep network for image compressive sensing," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1828–1837, 2018.

[2] J. Xiang, Y. Dong, and Y. Yang, "Fista-net: Learning a fast iterative shrinkage thresholding network for inverse problems in imaging," *IEEE Transactions on Medical Imaging*, vol. 40, no. 5, pp. 1329–1341, 2021.

[3] K. H. Jin, M. T. McCann, E. Froustey, and M. Unser, "Deep convolutional neural network for inverse problems in imaging," *IEEE Transactions on Image Processing*, vol. 26, no. 9, pp. 4509–4522, 2017.

[4] K. Hammernik, T. Klatzer, E. Kobler, M. P. Recht, D. K. Sodickson, T. Pock, and F. Knoll, "Learning a variational network for reconstruction of accelerated mri data," *Magnetic Resonance in Medicine*, vol. 79, no. 6, pp. 3055–3071, 2018.

[5] M. Garland, S. Le Grand, J. Nickolls, J. Anderson, J. Hardwick, S. Morton, E. Phillips, Y. Zhang, and V. Volkov, "Parallel computing experiences with cuda," *IEEE micro*, vol. 28, no. 4, pp. 13–27, 2008.